Code walkthrough: fdrm_vix_analysis.py

0. How to run it, and the map of files

The whole analysis is one Python script of 726 lines, `fdrm_vix_analysis.py`. It reads two CSV files, prints its key numbers as it goes, writes every table to `output/`, every chart to `figures/` and every quoted number to `output/results.json`. The deck and the report were written from those files. This tab goes through the script block by block, quoting the code and explaining each line.

Run it.

```
pip install pandas numpy scipy statsmodels matplotlib
cd FDRM_VIX
python fdrm_vix_analysis.py
```

It runs in about 30 seconds, most of it the 70-odd backtests in the robustness section. Re-running it from scratch reproduces `results.json` exactly; that was checked while writing this guide. It was built with Python 3.11, pandas 3.0, NumPy 2.4, SciPy 1.17, statsmodels 0.15 and matplotlib 3.11. Older versions should work, but if a number differs, check versions first.

The notebook. `fdrm_vix_analysis.ipynb` is the same code split into cells, already run, with every printout and chart visible. Open it in Jupyter or VS Code to read the outputs without running anything. The script marks cell boundaries with `# %%` comments and text cells with `# %% [markdown]`. That is the "percent" format used by the jupytext tool, which turned the script into the notebook. If you change the code, change the `.py`, re-run it, and regenerate the notebook with `jupytext --to ipynb fdrm_vix_analysis.py` followed by `jupyter nbconvert --to notebook --execute --inplace fdrm_vix_analysis.ipynb`.

What the script produces, by section.

Section	Output files
1. Load and align	`output/nifty_vix_merged.csv` (the cleaned, joined daily data with every derived column)
2. VIX formula check	Numbers only (in `results.json`)
3. VRP	`output/vrp_by_regime.csv`, `figures/fig2_vrp.png`

Section	Output files
4. Regimes	`output/returns_by_regime.csv`, `output/sameday_by_regime.csv`, `figures/fig1_nifty_vix_regimes.png`, `figures/fig3_regime_returns.png`
5. Election	`output/election_window.csv`, `output/event_comparison.csv`, `figures/fig4_election_window.png`, `figures/fig5_put22500.png`
6. Backtest	`output/backtest_values.csv` (daily value of all 7 strategies), `output/backtest_performance.csv`, `output/ledger_s1..s6_*.csv` (every option trade), `output/hedge_cost_payoff_by_regime.csv`, `output/excess_return_by_regime_*.csv`, `output/episode_returns.csv`, `figures/fig6_backtest_equity.png`, `figures/fig7_cost_by_regime.png`
7. Robustness	`output/sensitivity_thresholds.csv`, `output/sensitivity_pricing.csv`, `output/sensitivity_rule_by_tenor.csv`, `figures/fig8_sensitivity.png`
End	`output/results.json`

A quirk worth knowing: the ledgers are numbered by the strategy's position in the list, so `ledger_s3_timed_hold.csv` is "Always 3M put" (which reuses the timed-hold code with the VIX filter switched off) and `ledger_s4_timed_hold.csv` is the real "VIX-timed, hold". The unhedged strategy has no trades, so there is no s0 ledger.

How to change something. Every assumption is a constant near the top (section 1 of this tab). Change it, re-run, and every table, chart and `results.json` updates. The report and the deck do not update themselves; their numbers would need re-copying.

1. Imports, folders and assumptions (lines 1 to 72)

The file opens with a markdown cell (lines 1 to 18) describing the data and the modelling choice. Then the imports:

```
import json                      # to write results.json
import math                      # sqrt, exp on single numbers
from pathlib import Path         # file paths that work on any OS

import matplotlib
matplotlib.use("Agg")            # draw charts to files, no screen needed
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np               # fast maths on whole arrays
import pandas as pd              # tables indexed by date
import statsmodels.api as sm     # regressions with Newey-West errors
from scipy import stats          # Kruskal-Wallis and Levene tests
from scipy.stats import norm     # normal distribution N(.) for Black-Scholes
```

`matplotlib.use("Agg")` must come before `pyplot` is imported. It picks a backend that renders straight to image files, so the script also runs on a server with no display.

```
ROOT = Path(__file__).resolve().parent if "__file__" in globals() else
Path.cwd()
DATA, FIG, OUT = ROOT / "data", ROOT / "figures", ROOT / "output"
FIG.mkdir(exist_ok=True)
OUT.mkdir(exist_ok=True)
```

- `ROOT` is the folder the script lives in. When the code runs as a script, `__file__` is its path. Inside a notebook `__file__` does not exist, so it falls back to the current working directory. This one line makes the same code work both ways.
- `/` on `Path` objects joins folder names, so `ROOT / "data"` is the data folder.
- `mkdir(exist_ok=True)` creates the output folders if missing and does nothing if they exist.

The assumptions block. Every modelling choice is a named constant, so it can be changed in one place and re-tested.

```
R = 0.065            # risk-free rate
Q = 0.013            # Nifty dividend yield
SKEW_PER_PCT = 0.6   # extra put IV per 1% OTM
HALF_SPREAD = 0.5    # bid/ask half-spread in vol points
LOW_VIX, HIGH_VIX = 13.0, 20.0
SPIKE_1, SPIKE_2 = 22.0, 30.0
TRAIL = 0.20
TENOR = 3
PUT_MONEYNESS = 0.95
SPREAD_SHORT = 0.85
TRADING_DAYS = 252
```

Constant	Value	Why this value
`R`	6.5%	About the 91-day Treasury bill yield over the period. Enters Black-Scholes and the Sharpe ratio.
`Q`	1.3%	Roughly Nifty's dividend yield. Lowers the forward price, which makes puts slightly dearer.
`SKEW_PER_PCT`	0.6	A 5% OTM put gets 3 extra vol points, in line with typical Nifty skew. Stress-tested from 0 to 1.0.
`HALF_SPREAD`	0.5	Buy at IV + 0.5, sell at IV − 0.5. Stress-tested from 0 to 1.0.
`LOW_VIX`, `HIGH_VIX`	13, 20	Regime cut-offs, near the 30th and 80th percentiles of VIX in the sample. `LOW_VIX` is also the buy threshold.
`SPIKE_1`	22	First monetisation trigger. VIX closed at or above 22 on about the top 15% of days.
`SPIKE_2`	30	Second rung, used only by the ladder rule.
`TRAIL`	0.20	Sell the rest when VIX is 20% below its post-spike peak.
`TENOR`	3	Months to expiry of the proposed put.
`PUT_MONEYNESS`	0.95	Strike = 95% of spot, so 5% OTM.
`SPREAD_SHORT`	0.85	Short leg of the put-spread baseline, 15% OTM.
`TRADING_DAYS`	252	Trading days per year, for annualising.

Chart styling (lines 53 to 64). `C` is a dictionary of named hex colours so every chart uses the same palette. `REGIME_COL` gives the three regime shading colours (blue for Low, grey for Normal, orange for High). `plt.rcParams.update({...})` sets defaults for all charts: resolution, font size, grey axes, no top and right borders, light grid, no legend frame. None of this affects any number.

```
RESULTS = {}

def save(fig, name):
    fig.tight_layout()
    fig.savefig(FIG / name, bbox_inches="tight", facecolor="white")
    plt.close(fig)
```

- `RESULTS` is an empty dictionary that collects every headline number as the script runs. At the end it is written to `results.json`. This is what keeps the report, the deck and the code consistent.
- `save` tidies the layout so labels don't overlap, writes the PNG with a white background, and closes the figure to free memory.

2. Loading and aligning the data (lines 77 to 102)

```
nifty = pd.read_csv(DATA / "nifty50_daily_raw.csv", parse_dates=
["Date"]).set_index("Date")
nifty = nifty[["Open", "High", "Low", "Close"]].astype(float)
nifty = nifty[~nifty.index.duplicated()].sort_index()
```

- `read_csv(..., parse_dates=["Date"])` reads the file and turns the Date column into real dates. `.set_index("Date")` makes the date the row label, so rows can be looked up by date and two tables can be joined on it.
- The second line keeps four columns (dropping Adj Close and Volume) and forces them to floating-point numbers.
- `nifty.index.duplicated()` is True for any repeated date; `~` flips it, so the third line keeps only the first copy of each date. `sort_index()` puts rows in date order. Neither changed anything for this file, but they protect against a messy download.

```
vix_raw = pd.read_csv(DATA / "india_vix_daily_raw.csv", encoding="utf-8-sig",
parse_dates=["Date"])
vix = vix_raw.rename(columns={"Sum of Open": "VIX_Open", "Sum of High":
"VIX_High",
                              "Sum of Low": "VIX_Low", "Sum of Close":
"VIX"})
vix = vix.set_index("Date")[["VIX_Open", "VIX_High", "VIX_Low",
"VIX"]].astype(float).sort_index()
```

- `encoding="utf-8-sig"` strips the invisible byte-order mark at the start of this file. Without it the first column would be called "Date" and `parse_dates=["Date"]` would fail.
- `rename` maps the Power BI-style names to short ones. The VIX close is simply called `VIX` because it is used everywhere.

```
nifty["ret"] = np.log(nifty.Close).diff()
nifty["rv21_trail"] = nifty.ret.rolling(21).std() * math.sqrt(TRADING_DAYS) *
100
```

- `ret` is the daily log return: the log of today's close minus the log of yesterday's. `.diff()` subtracts the previous row. The first row becomes NaN (not a number) because it has no previous day.
- `rv21_trail` is trailing realised volatility: `.rolling(21).std()` is the standard deviation of the last 21 returns up to and including today. Multiplying by √252 annualises it, and by 100 puts it in percent, the same units as VIX.

```
nifty["rv21_fwd"] = nifty.ret[::-1].rolling(21).std()[::-1].shift(-1) *
math.sqrt(TRADING_DAYS) * 100
```

The most important line in this section. It computes the volatility over the **next** 21 trading days, which is what VIX on day t is trying to forecast. pandas has no forward-looking rolling window, so the line uses a trick:

1. `nifty.ret[::-1]` reverses the series so the latest date comes first.
2. `.rolling(21).std()` on the reversed series covers, for each date t, the return on t and the 20 returns after it (in normal time order).
3. `[::-1]` flips it back to normal order.
4. `.shift(-1)` moves each value up by one row, so the value at t now covers returns t+1 to t+21. Today's return is excluded, because VIX at today's close already knows it.
5. `× √252 × 100` annualises and converts to percent.

The last 21 rows have no full future window, so they are NaN and drop out of the VRP analysis.

```
nifty["fwd21"] = nifty.Close.shift(-21) / nifty.Close - 1
nifty["fwd63"] = nifty.Close.shift(-63) / nifty.Close - 1
nifty["fwd1"] = nifty.ret.shift(-1)
```

- `shift(-21)` pulls the close from 21 rows later onto today's row. Dividing by today's close and subtracting 1 gives the simple return over the next 21 trading days. Same for 63 days (about three months).
- `fwd1` is tomorrow's log return.

All of these are computed on the full Nifty history before the join. If they were computed after, the 16 days missing from the VIX file would make some "one-day" returns span two days.

```
df = nifty.join(vix, how="inner").dropna(subset=["Close", "VIX"])
df["dvix"] = np.log(df.VIX).diff()
df["regime"] = pd.cut(df.VIX, [0, LOW_VIX, HIGH_VIX, 1e9], labels=["Low",
"Normal", "High"])
df.to_csv(OUT / "nifty_vix_merged.csv")
```

- `join(..., how="inner")` matches rows by date and keeps only dates present in both files: 1,536 days from 2 Jan 2020 to 13 Apr 2026. Nifty data from before 2020 is dropped here because there is no VIX for it.
- `dvix` is the daily change in log VIX, roughly the percentage change.
- `pd.cut` puts each day's VIX into a bin. The edges [0, 13, 20, 1e9] (1e9 is just "a very large number") create three bins labelled Low, Normal and High. One detail for inside-out knowledge: bins include their right edge, so a VIX of exactly 13.00 is labelled Low and 20.00 Normal. The backtest's buy rule uses a strict `VIX < 13`. With prices to two decimals, this almost never matters.
- The merged table is saved, so you can open it in Excel and see every derived column.

```
RESULTS["sample"] = dict(start=..., end=..., days=len(df),
                         vix_pct=df.VIX.quantile([.1, .3, .5, .8,
.9]).round(2).to_dict(),

regime_share=df.regime.value_counts(normalize=True).round(3).to_dict())
```

This stores the sample dates, VIX percentiles (10th 11.4, 30th 13.4, median 15.3, 80th 21.0, 90th 24.1) and the share of days in each regime (Low 27%, Normal 48%, High 25%). The percentiles are how the 13 and 20 cut-offs were justified.

3. Black-Scholes functions and the India VIX formula check (lines 115 to 158)

```
def bs_put(S, K, T, sigma, r=R, q=Q):
    T = np.maximum(T, 1e-8)
    sigma = np.maximum(sigma, 1e-4)
    d1 = (np.log(S / K) + (r - q + 0.5 * sigma ** 2) * T) / (sigma *
np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * np.exp(-q * T) *
norm.cdf(-d1)
```

- This is the Black-Scholes-Merton put formula from Theory 3, written line for line. `norm.cdf` is N(·), the standard normal cumulative probability.
- The two `np.maximum` lines are guards. A time of exactly zero or a volatility of zero would mean dividing by zero, so they are floored at tiny positive numbers.
- Every operation uses NumPy, so the function works on one number or on whole arrays at once. The election code prices a whole column of dates in one call because of this.
- `r=R, q=Q` are default arguments: if you don't pass a rate or dividend yield, it uses the constants from the top.

```
def bs_call(S, K, T, sigma, r=R, q=Q):
    return bs_put(S, K, T, sigma, r, q) + S * np.exp(-q * T) - K * np.exp(-r
* T)
```

The call is the put plus S·e^(−qT) − K·e^(−rT): put-call parity rearranged. Writing it this way guarantees calls and puts are consistent.

```
def put_iv(vix_level, S, K):
    otm_pct = np.maximum(0.0, (1 - K / S) * 100)
    return (vix_level + SKEW_PER_PCT * otm_pct) / 100
```

The skew model. `(1 − K/S) × 100` is how many percent the strike is below spot. `np.maximum(0, …)` makes it zero for strikes at or above spot. The result is VIX + 0.6 per OTM percent, divided by 100 to turn vol points into a decimal (15 becomes 0.15). The election study uses this; the backtest has its own version with the spread added (section 7).

Re-implementing India VIX.

```
def model_free_var(S, T, iv_fn, r=R, q=Q, step=50):
    F = S * math.exp((r - q) * T)
    Ks = np.arange(round(S * 0.6 / step) * step, round(S * 1.4 / step) * step
+ step, step, dtype=float)
    K0 = Ks[Ks <= F].max()
    Qk = np.where(Ks < K0, bs_put(S, Ks, T, iv_fn(Ks)),
                  np.where(Ks > K0, bs_call(S, Ks, T, iv_fn(Ks)),
                           0.5 * (bs_put(S, Ks, T, iv_fn(Ks)) + bs_call(S,
Ks, T, iv_fn(Ks)))))
    dK = np.gradient(Ks)
    return (2 / T) * np.sum(dK / Ks ** 2 * math.exp(r * T) * Qk) - (1 / T) *
(F / K0 - 1) ** 2
```

This computes the variance for one expiry, following NSE's steps.

- `F` is the forward level. NSE derives it from put-call parity on real quotes; on a synthetic chain it is simply S·e^((r − q)T).
- `Ks` is the strike grid: from 60% to 140% of spot, 50 points apart, like real Nifty strikes. `round(S × 0.6 / 50) × 50` rounds to the nearest 50. `np.arange` excludes its end point, hence the `+ step`.
- `K0` is the highest strike at or below the forward.
- `Qk` picks the option for each strike with nested `np.where(condition, if_true, if_false)`: a put below K0, a call above K0, and the average of both at K0. These are the out-of-the-money options the formula uses. `iv_fn(Ks)` supplies a volatility for every strike, which is how the smile is fed in.

- `dK` is the strike spacing ΔK. `np.gradient` takes half the distance between each strike's two neighbours, which is exactly how the CBOE and NSE define ΔK. Here it is 50 everywhere.
- The return line is the formula from Theory 2: (2/T) × Σ ΔK/K² · e^(RT) · Q(K), minus the small correction (1/T)(F/K0 − 1)².

```
def india_vix_formula(S, iv_fn, d_near=16, d_next=44):
    T1, T2 = d_near / 365, d_next / 365
    v1, v2 = model_free_var(S, T1, iv_fn), model_free_var(S, T2, iv_fn)
    N30 = 30 / 365
    w = (T2 - N30) / (T2 - T1)
    return 100 * math.sqrt((T1 * v1 * w + T2 * v2 * (1 - w)) / N30)
```

- Two expiries: a near month with 16 days left and a next month with 44, a typical mid-month situation.
- `w` is the weight on the near expiry. It is the share of the gap between T1 and T2 that lies beyond 30 days: (44 − 30) ÷ (44 − 16) = 0.5.
- The interpolation blends *total* variance (variance × time), not variance itself, then divides by 30 days to get back a 30-day annualised variance. This is the CBOE/NSE interpolation in years instead of minutes.
- `100 × sqrt(…)` turns the variance into an index level.

```
S0 = 24000.0
flat = india_vix_formula(S0, lambda K: np.full_like(K, 0.15))
skew = india_vix_formula(S0, lambda K: 0.15 + 0.006 * np.maximum(0, (1 - K /
S0) * 100)
                         - 0.002 * np.maximum(0, (K / S0 - 1) * 100))
```

- `lambda K: …` is a one-line unnamed function. Each defines a smile: volatility as a function of strike.
- `flat`: every strike at 15%. Under Black-Scholes with constant volatility, the fair variance is exactly σ², so a correct implementation must return 15. It returns **15.0**, which validates the code.
- `skew`: 15% at the money, +0.6 vol points per 1% below spot for puts, and −0.2 per 1% above spot for calls (the usual smirk shape). It returns **16.1**. The extra 1.1 points come from the 1/K² weighting on expensive low-strike puts, which shows that VIX prices crash protection as well as expected volatility.

4. Volatility risk premium (lines 165 to 196)

```
v = df.dropna(subset=["rv21_fwd"]).copy()
v["vrp"] = v.VIX - v.rv21_fwd
```

- Keep only days that have a full 21-day future window (this drops the last 21 days). `.copy()` makes `v` an independent table so adding columns to it never touches `df`.
- `vrp` is the volatility risk premium for each day: VIX minus the volatility that actually followed. Both are in annualised percent, so the difference is in vol points.

```
vrp_stats = dict(
    mean_vix=v.VIX.mean(), mean_rv_fwd=v.rv21_fwd.mean(),
mean_vrp=v.vrp.mean(), median_vrp=v.vrp.median(),
    share_positive=(v.vrp > 0).mean(),
    corr_vix_rvfwd=v[["VIX", "rv21_fwd"]].corr().iloc[0, 1],
    corr_vix_rvtrail=v[["VIX", "rv21_trail"]].corr().iloc[0, 1],
)
```

- Means and median of VIX, forward RV and VRP (17.3, 15.1, 2.2 and 3.0).
- `(v.vrp > 0)` is a column of True/False. The mean of True/False is the share of True, here 0.78.
- `.corr()` returns a 2×2 correlation matrix; `.iloc[0, 1]` picks the off-diagonal number. VIX correlates 0.61 with future RV and 0.86 with past RV.

```
nw = sm.OLS(v.vrp.values, np.ones(len(v))).fit(cov_type="HAC", cov_kwds=
{"maxlags": 21})
vrp_stats["nw_t_mean_vrp"] = float(nw.tvalues[0])
```

The Newey-West trick from Theory 4. `sm.OLS(y, X)` sets up a least-squares regression. Here X is a column of ones, so the only coefficient is the mean of the VRP. `.fit(cov_type="HAC", cov_kwds={"maxlags": 21})` estimates it with standard errors robust to 21 days of overlap. `tvalues[0]` is the t-statistic of the mean: 2.8.

```
by_reg = v.groupby("regime", observed=True).agg(VIX=("VIX", "mean"), RV_fwd=
("rv21_fwd", "mean"),
                                                VRP=("vrp", "mean"), VRP_pos=
("vrp", lambda x: (x > 0).mean()),
                                                days=("vrp", "size"))
by_reg.round(3).to_csv(OUT / "vrp_by_regime.csv")
```

- `groupby("regime")` splits the days into Low, Normal and High. `observed=True` tells pandas to use only the categories that actually appear, which avoids a warning with categorical columns.

- `.agg(name=(column, function))` is "named aggregation": each argument creates one output column. For example `VRP=("vrp", "mean")` is the mean VRP per regime, and `VRP_pos` uses a small lambda to get the share of positive days. `"size"` counts the rows.
- The result is the VRP-by-regime table (2.1, 1.4 and 4.0 points).

```
mz = sm.OLS(v.rv21_fwd, sm.add_constant(v.VIX)).fit(cov_type="HAC", cov_kwds=
{"maxlags": 21})
vrp_stats.update(mz_alpha=float(mz.params.iloc[0]),
mz_beta=float(mz.params.iloc[1]), mz_r2=float(mz.rsquared))
```

The Mincer-Zarnowitz regression: future RV on VIX. `sm.add_constant` adds the column of ones for the intercept. `params` holds the intercept (1.05) and slope (0.81), and `rsquared` is 0.38. HAC errors are used again because the dependent variable overlaps.

```
RESULTS["vrp"] = {k: round(float(x), 3) for k, x in vrp_stats.items()}
RESULTS["vrp_by_regime"] =
by_reg.round(2).reset_index().to_dict(orient="records")
```

A dictionary comprehension rounds every statistic and stores it. `reset_index()` turns the regime labels back into a normal column, and `to_dict(orient="records")` turns the table into a list of rows, which is how tables are stored in `results.json`.

The chart (fig2_vrp.png). `plt.subplots(2, 1, sharex=True, gridspec_kw={"height_ratios": [2, 1]})` makes two stacked panels sharing a date axis, the top one twice as tall. The top panel plots VIX (blue) and forward RV (orange). The bottom panel uses `fill_between(…, where=v.vrp >= 0)` to shade the VRP blue where VIX was above what followed and red where it was below. The y-axis is cut at −35, so the deepest red (−64 in early March 2020) is clipped. `save(fig, "fig2_vrp.png")` writes it.

5. VIX regimes and Nifty returns (lines 202 to 273)

```
reg_rows = []
for reg, g in df.groupby("regime", observed=True):
    for h, col in [("1d", "fwd1"), ("21d", "fwd21"), ("63d", "fwd63")]:
        x = g[col].dropna()
        lag = {"1d": 1, "21d": 21, "63d": 63}[h]
        m = sm.OLS(x.values, np.ones(len(x))).fit(cov_type="HAC", cov_kwds=
{"maxlags": lag})
        reg_rows.append(dict(regime=reg, horizon=h, n=len(x),
mean_pct=x.mean() * 100, median_pct=x.median() * 100,
                             sd_pct=x.std() * 100, hit_rate=(x > 0).mean(),
nw_t=float(m.tvalues[0]),
                             p5_pct=x.quantile(.05) * 100))
reg_tab = pd.DataFrame(reg_rows)
```

- The outer loop gives each regime's name `reg` and its sub-table `g`. The inner loop goes over three horizons, pairing a label with the column holding that forward return.
- `x` is the forward returns for that regime and horizon, without the NaNs at the end of the sample.
- `lag` looks up the Newey-West lag: 1 for daily, 21 and 63 for the overlapping horizons.
- The OLS-on-a-constant trick again gives the Newey-West t-statistic of the mean.
- Each combination becomes one dictionary of statistics, appended to a list. `pd.DataFrame(reg_rows)` turns the list into the 9-row table saved as `returns_by_regime.csv`. Building a list of dicts and converting once at the end is the standard, fast pattern in pandas.

```
same_day = df.groupby("regime", observed=True).ret.agg(
    mean_pct=lambda x: x.mean() * 100, sd_pct=lambda x: x.std() * 100,
    ann_vol=lambda x: x.std() * math.sqrt(TRADING_DAYS) * 100,
    worst_pct=lambda x: x.min() * 100, days="size")
```

Same-day statistics: each day's own return, grouped by that day's regime. This is where "9% volatility in Low, 13% in Normal, 30% in High" and the −13.9% worst day come from.

```
kw21 = stats.kruskal(*[g.fwd21.dropna() for _, g in df.groupby("regime",
observed=True)])
lev21 = stats.levene(*[g.fwd21.dropna() for _, g in df.groupby("regime",
observed=True)])
kw1 = stats.kruskal(*[g.ret.dropna() for _, g in df.groupby("regime",
observed=True)])
lev1 = stats.levene(*[g.ret.dropna() for _, g in df.groupby("regime",
observed=True)])
```

- The list comprehension builds a list of three series, one per regime. `_` is the Python convention for a value you don't need (the regime name).
- `*` unpacks the list, so `stats.kruskal(*[a, b, c])` is the same as `stats.kruskal(a, b, c)`. Both tests take any number of groups this way.
- `kw1` and `lev1` are the tests quoted in the report (same-day returns: p = 0.66 and p < 0.001). `kw21` and `lev21` use overlapping returns, so their p-values overstate significance.

```
xy = df[["ret", "dvix"]].dropna()
corr = xy.corr().iloc[0, 1]
up, dn = xy[xy.ret > 0], xy[xy.ret < 0]
beta = sm.OLS(xy.dvix, sm.add_constant(xy.ret)).fit()
pred = sm.OLS(df.fwd21.dropna(), sm.add_constant(df.loc[df.fwd21.notna(),
"VIX"])).fit(
    cov_type="HAC", cov_kwds={"maxlags": 21})
```

- `corr`: daily Nifty return against daily log-VIX change, −0.56.
- `up` and `dn` filter rows with boolean masks: `xy[xy.ret > 0]` keeps only up days. Their mean `dvix` gives the asymmetry (−1.9% on up days, +2.3% on down days).
- `beta`: regression of the VIX change on the Nifty return, slope −2.7. Daily changes do not overlap, so plain standard errors are fine here.
- `pred`: next-21-day return on today's VIX. `df.loc[df.fwd21.notna(), "VIX"]` selects VIX on exactly the rows where the forward return exists, so the two sides line up. HAC errors with 21 lags because the returns overlap. Slope 0.18% per VIX point, t = 4.3.

The next lines store all of this in `RESULTS`, rounding each value with a dictionary comprehension.

Figure 1 (Nifty and VIX shaded by regime). The interesting part is the shading:

```
seg = (df.regime != df.regime.shift()).cumsum()
for _, g in df.groupby(seg):
    for a in ax:
        a.axvspan(g.index[0], g.index[-1] + pd.Timedelta(days=1),
color=REGIME_COL[str(g.regime.iloc[0])], lw=0, zorder=0)
```

- `df.regime != df.regime.shift()` is True on each day the regime changes. `.cumsum()` counts the changes so far, which gives every unbroken run of the same regime its own number.
- Grouping by that number gives one group per run. `axvspan` shades the run from its first to its last day (plus one day to close gaps) in that regime's colour, on both panels. `zorder=0` keeps the shading behind the lines.

- The two dashed lines are the 13 and 20 cut-offs. The event labels use `df.VIX.asof(d)`, which returns VIX on date d or the last trading day before it.

Figure 3 (regime returns and the leverage effect).

```
err = 1.96 * t21.mean_pct / t21.nw_t
ax[0].errorbar(range(3), t21.mean_pct, yerr=err.abs(), fmt="none",
ecolor=C["ink"], capsize=4, lw=1)
```

The left panel is a bar chart of mean next-21-day returns by regime. Since t = mean ÷ standard error, the standard error is mean ÷ t, and 1.96 standard errors gives the 95% Newey-West confidence interval drawn as error bars. The right panel is a scatter of daily Nifty returns against VIX changes with the fitted `beta` line drawn through it.

6. The election event study (lines 281 to 383)

```
def straddle_pct(vix_level, days=30):
    T = days / 365; s = vix_level / 100
    return (bs_put(1.0, 1.0, T, s) + bs_call(1.0, 1.0, T, s)) * 100

df["straddle30"] = straddle_pct(df.VIX)
df["implied_daily_move"] = df.VIX / math.sqrt(365) * math.sqrt(365 /
TRADING_DAYS)
df["abs_ret"] = df.ret.abs() * 100
```

- `straddle_pct` prices an ATM put plus an ATM call with spot and strike both set to 1.0. The answer is then automatically a fraction of spot; × 100 makes it a percentage. Because `bs_put` works on arrays, passing the whole VIX column prices a straddle for every day in one call.
- `implied_daily_move` is VIX converted to a one-day standard deviation. Dividing by √365 gives a calendar-day figure; multiplying by √(365/252) converts it to a trading day. The two steps simplify to VIX ÷ √252, written out to show the logic.
- `abs_ret` is the size of each day's move in percent, ignoring direction.

```
win = df.loc["2024-04-01":"2024-07-15"].copy()
exp_jun = pd.Timestamp("2024-06-27")
Kp = 22500.0
Tj = pd.Series(np.clip((exp_jun - win.index).days, 0, None) / 365,
index=win.index)
win["put22500"] = np.where(win.index <= exp_jun, bs_put(win.Close.values, Kp,
Tj.values,

put_iv(win.VIX.values, win.Close.values, Kp)), np.nan)
```

- `df.loc["2024-04-01":"2024-07-15"]` slices rows by date; pandas includes both ends.
- `exp_jun` is the June 2024 monthly expiry, Thursday 27 June. `Kp` is the strike.
- `(exp_jun - win.index).days` is the number of calendar days to expiry for every date in the window. `np.clip(…, 0, None)` turns negative values (dates after expiry) into 0. Dividing by 365 gives T in years.
- `np.where(condition, value, np.nan)` prices the put with Black-Scholes and the skewed IV up to expiry and leaves it blank afterwards. This produces the 409, 545, 217, 903, 335, 112 path.

```
base = df.loc["2024-01-01":"2024-03-31", "VIX"].median()
ev = {}
for d in ["2024-05-31", "2024-06-03"]:
    pre = float(df.VIX.loc[d])
    ev[d] = dict(vix=pre,
                 implied_event_sd_pct=round(math.sqrt(max((pre ** 2 * 30 -
base ** 2 * 29) / 365, 0)), 2),
                 vix_only_daily_sd_pct=round(pre / math.sqrt(TRADING_DAYS),
2))
```

- `base` is the "normal" VIX level, the median for January to March 2024 (14.41).
- For the last close before the exit polls and the last close before the result, the loop computes the event-day standard deviation with the variance formula from the study guide: $\sqrt{((\text{VIX_pre}^2 \times 30 - \text{base}^2 \times 29) \div 365)}$. `max(…, 0)` guards against a negative number under the square root if VIX were below base. Because VIX is in percent, the answer is directly in percent: 5.77 and 4.42.
- `vix_only_daily_sd_pct` is the naive VIX ÷ $\sqrt{252}$ for comparison: 1.55 and 1.32.

```
e = df.loc["2024-05-27":"2024-06-07", ["Close", "ret", "VIX", "straddle30",
"implied_daily_move"]].copy()
e["ret_pct"] = (np.exp(e.ret) - 1) * 100
e["put22500"] = win.put22500.reindex(e.index)
e.round(3).to_csv(OUT / "election_window.csv")
intraday_low = (df.Low.loc["2024-06-04"] / df.Close.loc["2024-06-03"] - 1) *
100
normal = df.loc["2024-01-01":"2024-04-30"]
```

- `e` is the two-week table around the event. `np.exp(ret) - 1` turns log returns into percentage moves (−5.93% on 4 June).
- `reindex(e.index)` copies the put price for exactly those dates.
- `intraday_low` compares the 4 June low with the 3 June close: −8.52%.

- `normal` is January to April 2024, the benchmark period for the medians (VIX 13.8, straddle 3.2%, daily move 0.56%).

The long `RESULTS["election"] = dict(…)` block stores all these numbers, including the day-by-day VIX, straddle and put paths as small dictionaries keyed by date (`str(k.date())` turns a timestamp into text like "2024-06-04").

The cross-event table.

```
for name, d in events.items():
    d = pd.Timestamp(d); i = df.index.get_loc(d)
    pre = df.iloc[i - 1]; row = df.iloc[i]
    rows.append(dict(event=name, date=str(d.date()), nifty_ret_pct=
(math.exp(row.ret) - 1) * 100,
                     vix_prev=pre.VIX, vix=row.VIX, vix_chg_pct=(row.VIX /
pre.VIX - 1) * 100,
                     straddle_prev_pct=pre.straddle30,
straddle_pct=row.straddle30,
                     move_vs_implied=abs(row.ret) * 100 /
pre.implied_daily_move))
```

- `df.index.get_loc(d)` finds the row number of the event date, so `iloc[i - 1]` is the previous trading day, whatever the calendar gap.
- Each event gets the Nifty move, VIX before and after, straddle before and after, and the move divided by the previous day's implied daily move.

```
nd = df[(df.abs_ret < df.abs_ret.quantile(.9))]
```

The "normal day" row uses every day except the biggest 10% of moves, and reports medians. `nd.implied_daily_move.shift(1)` uses the previous day's implied move, consistent with the event rows.

```
top = df.dvix.nlargest(20).index
```

`nlargest(20)` picks the 20 days with the biggest rise in log VIX; `.index` gives their dates. The block then averages the Nifty return, the VIX jump and the jump in the straddle (`df.straddle30 - df.straddle30.shift()`) on those days, and the average absolute move on all other days (`df.drop(top)`).

Figures 4 and 5. Figure 4 stacks Nifty, VIX (with the January–April median as a dashed line) and the straddle, with 4 June shaded using `axvspan` from 12 hours before to 12 hours after. Figure 5 plots the 22,500 put's modelled premium; `mdates.DateFormatter("%d-%b")` formats the dates as "04-Jun".

7. The backtest engine `run()`, line by line (lines 405 to 508)

This is the heart of the project. One function simulates any of the strategies, day by day, and returns the daily portfolio value plus a ledger of every option trade.

Setting up the calendar

```
bt = df.loc[:, ["Close", "VIX", "regime"]].copy()
dates = bt.index
first_of_month = pd.Series(dates, index=dates).groupby([dates.year,
dates.month]).transform("min")
roll_dates = set(first_of_month.unique())
roll_list = sorted(roll_dates)
```

- `bt` is the backtest's input: just Nifty close, VIX and regime for each day.
- `first_of_month` groups the dates by (year, month) and, with `transform("min")`, gives every date the earliest date in its month. `unique()` leaves one date per month: the first trading day. Those are the roll dates.
- `roll_dates` is a `set`, which makes "is today a roll date?" an instant check. `roll_list` is the same dates sorted, so they can be counted forwards.

```
def next_roll(d, k):
    i = roll_list.index(d)
    return roll_list[i + k] if i + k < len(roll_list) else None
```

The expiry date of an option bought on roll date `d` with a `k`-month tenor: the roll date `k` positions later. If that is beyond the end of the data it returns `None`, and no option is opened.

The function signature

```
def run(strategy, low=LOW_VIX, spike1=SPIKE_1, spike2=SPIKE_2, tenor=TENOR,
moneyness=PUT_MONEYNESS,
        skew=SKEW_PER_PCT, half_spread=HALF_SPREAD, tranche=1.0,
stagger=False, trail=TRAIL):
```

`strategy` is a text code: `"unhedged"`, `"always_put"`, `"always_spread"`, `"timed_hold"`, `"timed_all"`, `"timed_ladder"`, `"timed_trail"` or `"timed_hybrid"`. Every other argument defaults to the constants at the top, so `run("timed_hybrid")` is the proposal and the robustness section changes one argument at a time, for example `run("timed_hybrid", tenor=2)`. `tranche` is the fraction of the portfolio each new put covers, and `stagger=True` keeps old puts instead of replacing them.

The state the simulation carries

```
    units = 1.0 / bt.Close.iloc[0]   # start with 1.0 of wealth in Nifty
    pos = []                          # dicts: K, exp, qty, sign(+1 long/-1
short), trade_id
    trades = []                       # hedge cashflow ledger
    rec = []
    tid = 0
```

- `units`: how many "units" of the index the portfolio owns. Starting wealth is 1.0, so units = 1 ÷ the first close. Portfolio equity on any day is units × close.
- `pos`: the list of open options. `trades`: the ledger. `rec`: one record per day. `tid`: a counter that numbers each new option.

Four helper functions inside `run()`

They are defined inside `run()` so they can see and change its variables.

```
    def iv(vixl, S, K, side):
        otm = max(0.0, (1 - K / S) * 100)
        return max((vixl + skew * otm + side * half_spread) / 100, 0.01)
```

The backtest's implied volatility: VIX + skew × percent OTM, plus `side × half_spread`. `side` is +1 when buying (pay the offer), −1 when selling (hit the bid) and 0 for a mid-market valuation. The floor of 1% stops a negative volatility.

```
    def price(p, d, S, vixl, side=0):
        T = (p["exp"] - d).days / 365
        if T <= 0:
            return max(p["K"] - S, 0.0)
        return float(bs_put(S, p["K"], T, iv(vixl, S, p["K"], side)))
```

The value of position `p` on date `d`. Time to expiry is in calendar days ÷ 365. On or after expiry it is the intrinsic value; otherwise Black-Scholes with the IV above.

```
    def close_pos(p, d, S, vixl, frac, reason, regime):
        nonlocal units
        q = p["qty"] * frac
        px = price(p, d, S, vixl, side=-1 if p["sign"] > 0 else +1)
        cash = p["sign"] * q * px
        units += cash / S
        p["qty"] -= q
        trades.append(dict(date=d, trade_id=p["tid"], action=reason,
K=p["K"], qty=q * p["sign"], price=px,
                           cash=cash, vix=vixl, regime=regime,
open_regime=p["open_regime"]))
```

Sells (or buys back) a fraction `frac` of a position.

- `nonlocal units` lets this inner function change the outer `units` variable.
- `q` is the quantity being closed: all of it with `frac=1.0`, half with 0.5.
- Price: a long put is sold at the bid (side −1); a short put is bought back at the offer (+1).
- `cash` is positive when selling a long put. `units += cash / S` buys Nifty with the proceeds the same day.
- The position's quantity is reduced, and the trade is written to the ledger with the reason ("expiry", "roll_sell", "monetise_1", "monetise_trail" and so on), the regime today and the regime when it was opened.

```
    def open_pos(d, S, vixl, K, months, sign, regime, frac=1.0):
        nonlocal units, tid
        exp = next_roll(d, months)
        if exp is None:
            return
        V = units * S + sum(price(p, d, S, vixl) * p["qty"] * p["sign"] for p
in pos)
        qty = frac * V / S
        p = dict(K=K, exp=exp, qty=qty, sign=sign, tid=tid,
open_regime=regime, sold1=False)
        px = price(p, d, S, vixl, side=+1 if sign > 0 else -1)
        cash = -sign * qty * px
        units += cash / S
        pos.append(p)
        trades.append(dict(date=d, trade_id=tid, action="open", K=K, qty=qty
* sign, price=px, cash=cash,
                           vix=vixl, regime=regime, open_regime=regime))
        tid += 1
```

Buys (sign +1) or sells (sign −1) a new put.

- The expiry comes from `next_roll`. If the data ends first, nothing is opened.

- `V` is today's total portfolio value: Nifty holdings plus any options still open, valued at mid.
- `qty = frac × V ÷ S` sizes the put so it covers `frac` of the whole portfolio (100% by default). Quantity is in index units, so a put on qty units pays qty × (K − S) at expiry.
- The new position records its strike, expiry, size, direction, id, the regime it was opened in, and `sold1=False` (the first half has not been monetised yet).
- Buying pays the offer. `cash` is negative, so `units += cash / S` sells Nifty units to pay the premium. Selling (the short leg of the spread) receives the bid and buys units.

The daily loop

```
    for d, row in bt.iterrows():
        S, vixl, regime = row.Close, row.VIX, str(row.regime)
```

`iterrows()` walks through the days in order. `S`, `vixl` and `regime` are today's close, VIX close and regime label.

Step 1: settle expiries.

```
        for p in pos:
            if p["qty"] > 0 and p["exp"] <= d:
                close_pos(p, d, S, vixl, 1.0, "expiry", regime)
        pos = [p for p in pos if p["qty"] > 1e-12]
        is_roll = d in roll_dates
```

Any option whose expiry has arrived is closed at intrinsic value (`price` returns max(K − S, 0) when T ⩽ 0). The list comprehension then drops positions with nothing left; `1e-12` avoids keeping tiny floating-point leftovers. Expiries are always roll dates, so on a roll date the old 1-month put settles first and the new one is bought after.

Step 2a: always-on strategies.

```
        if strategy == "always_put" and is_roll:
            open_pos(d, S, vixl, round(S * moneyness / 50) * 50, 1, +1,
regime)
        elif strategy == "always_spread" and is_roll:
            open_pos(d, S, vixl, round(S * moneyness / 50) * 50, 1, +1,
regime)
            open_pos(d, S, vixl, round(S * SPREAD_SHORT / 50) * 50, 1, -1,
regime)
```

On every roll date: buy a 1-month put at 95% of spot, rounded to the nearest 50-point strike. The spread also sells a 1-month 85% put.

Step 2b: timed strategies, monetisation first.

```
        elif strategy.startswith("timed"):
            monetise = strategy in ("timed_ladder", "timed_all",
"timed_trail", "timed_hybrid")
            if monetise:
                for p in pos:
                    if strategy in ("timed_trail", "timed_hybrid"):
                        if vixl >= spike1:
                            p["peak"] = max(p.get("peak", 0.0), vixl)
                        if strategy == "timed_hybrid" and not p["sold1"] and
vixl >= spike1:
                            close_pos(p, d, S, vixl, 0.5, "monetise_1",
regime); p["sold1"] = True
                        if p.get("peak") and vixl <= (1 - trail) * p["peak"]
and p["qty"] > 0:
                            close_pos(p, d, S, vixl, 1.0, "monetise_trail",
regime)
                    elif strategy == "timed_ladder":
                        if not p["sold1"] and vixl >= spike1:
                            close_pos(p, d, S, vixl, 0.5, "monetise_1",
regime); p["sold1"] = True
                        if p["sold1"] and vixl >= spike2 and p["qty"] > 0:
                            close_pos(p, d, S, vixl, 1.0, "monetise_2",
regime)
                    elif vixl >= spike1:
                        close_pos(p, d, S, vixl, 1.0, "monetise", regime)
                pos = [p for p in pos if p["qty"] > 1e-12]
```

`timed_hold` skips this block entirely: it never sells early. For the others, each open put is checked against today's VIX close.

- **Trailing and hybrid.** Once VIX closes at or above 22, the put is "armed" and remembers its `peak`, the highest VIX close at or above 22 so far. `p.get("peak", 0.0)` returns 0 if no peak is stored yet. The hybrid also sells half on the first such day and sets `sold1` so it never does that again. Then, on any later day when VIX has fallen to 80% of the peak or below, the rest is sold. `p.get("peak")` is `None` until armed, so the fade check cannot fire before a spike. It also cannot fire on the arming day, since the peak is then at least today's VIX. `timed_trail` is the same without the half sale.
- **Ladder.** Half at 22, the rest at 30. If VIX never reaches 30, the second half is held to expiry.
- **Sell all** (`timed_all`) falls to the last `elif`: everything at 22.

Step 2c: timed strategies, buy or roll.

```
            if is_roll and vixl < low:
                if not stagger:
                    for p in pos:
                        close_pos(p, d, S, vixl, 1.0, "roll_sell", regime)
                    pos = []
                open_pos(d, S, vixl, round(S * moneyness / 50) * 50, tenor,
+1, regime, frac=tranche)
```

Only on a roll date with VIX below the threshold. In the normal version the old put (or whatever half of it is left) is sold at the bid and a fresh `tenor`-month 95% put is bought for the whole portfolio. If VIX is 13 or above, nothing happens and the existing put is simply held. In the staggered version old puts are kept and a new tranche (a third or a sixth of the portfolio) is added. Note the trick that defines the "Always 3M" baseline: `run("timed_hold", low=1e9)`. With a threshold of a billion, VIX is always "low", so it rolls every month.

Step 3: record the day.

```
        hedge_mtm = sum(price(p, d, S, vixl) * p["qty"] * p["sign"] for p in
pos)
        rec.append(dict(date=d, equity=units * S, hedge=hedge_mtm,
value=units * S + hedge_mtm,
                        notional_hedged=sum(p["qty"] * p["sign"] for p in pos
if p["sign"] > 0) * S))
    out = pd.DataFrame(rec).set_index("date")
    return out, pd.DataFrame(trades)
```

The options are valued at mid, the day's total value is equity plus options, and the hedged notional is recorded (used later for "time hedged"). After the last day, the records become a table indexed by date, returned with the ledger.

One trade traced through the code

The proposal's first trade, from `ledger_s5_timed_hybrid.csv`:

1. **2 January 2020**, a roll date, VIX 11.49 < 13. `open_pos` buys a put with strike 11,650 (95% of spot, rounded) expiring on `next_roll(d, 3)` = 1 April 2020. Quantity 0.0000814 units = the whole portfolio ÷ spot. Price 92.86 points, cash −0.0076, so 0.76% of the portfolio is sold to pay for it.
2. **3 February 2020**, a roll date, but VIX is above 13, so nothing happens. The put is held.
3. **28 February 2020**, VIX closes at 23.24 ⩾ 22. The put is armed with peak 23.24, and `close_pos(…, 0.5, "monetise_1")` sells half at the bid for 545.84 points. Cash +0.0222 buys Nifty units.

4. **March 2020**, VIX keeps rising, so the peak keeps rising (to the 83.6 close on 24 March). The fade rule needs VIX at or below 0.8 × peak, about 67.
5. **31 March 2020**, VIX closes at 64.41, more than 20% below its peak. `close_pos(…, 1.0, "monetise_trail")` sells the remaining half for 3,050.48 points, cash +0.1242.

The trade cost 0.76% of the starting portfolio and brought back 14.6% of it. That single trade is most of the proposal's advantage over the whole six years.

8. Metrics, running the strategies, cost by regime and episodes (lines 511 to 648)

The metrics function

```
def metrics(val, rf=R):
    r = val.pct_change().dropna()
    yrs = (val.index[-1] - val.index[0]).days / 365.25
    cagr = (val.iloc[-1] / val.iloc[0]) ** (1 / yrs) - 1
    vol = r.std() * math.sqrt(TRADING_DAYS)
    dd = val / val.cummax() - 1
    downside = r[r < 0].std() * math.sqrt(TRADING_DAYS)
    return dict(final=val.iloc[-1], CAGR=cagr * 100, vol=vol * 100, sharpe=
(cagr - rf) / vol,
                sortino=(cagr - rf) / downside, maxDD=dd.min() * 100,
calmar=cagr / abs(dd.min()),
                worst_day=r.min() * 100, worst_21d=(val / val.shift(21) -
1).min() * 100)
```

Takes a series of daily portfolio values and returns the performance numbers defined in the study guide.

- `pct_change()` gives daily simple returns. `yrs` is the length in years, using 365.25 days to allow for leap years.
- `cagr` is (end ÷ start) to the power 1 ÷ years, minus 1.
- `val.cummax()` is the running highest value so far. `val / val.cummax() - 1` is the drawdown on each day (0 at a new high, negative below it). Its minimum is the maximum drawdown.
- `downside` is the standard deviation of only the negative daily returns, annualised, for the Sortino ratio.
- `val / val.shift(21) - 1` is the return over every 21-day window; its minimum is the worst month.

Running all seven strategies

```
PROPOSED = "VIX-timed 3M put + hybrid monetisation (proposed)"
STRATS = {"Unhedged": ("unhedged", {}), "Always 1M 95% put": ("always_put",
{}),
          "Always 1M 95/85 put spread": ("always_spread", {}),
          "Always 3M 95% put, rolled monthly": ("timed_hold", {"low": 1e9}),
          "VIX-timed 3M put, hold": ("timed_hold", {}),
          PROPOSED: ("timed_hybrid", {}),
          "VIX-timed 3M put, sell all at VIX 22": ("timed_all", {})}
SLUG = {n: f"s{i}_{s}" for i, (n, (s, _)) in enumerate(STRATS.items())}
runs, ledgers = {}, {}
for name, (s, kw) in STRATS.items():
    runs[name], ledgers[name] = run(s, **kw)
vals = pd.DataFrame({k: r.value for k, r in runs.items()})
```

- `STRATS` maps each display name to the strategy code and any extra arguments. "Unhedged" works because `run` with an unknown strategy code simply never trades.
- `SLUG` builds short file names (`s5_timed_hybrid` and so on) from each strategy's position and code.
- `run(s, **kw)` unpacks the dictionary of extra arguments, so `{"low": 1e9}` becomes `run("timed_hold", low=1e9)`.
- `vals` is one table with a column per strategy and a row per day: the data behind the equity chart, saved as `backtest_values.csv`.

```
perf = pd.DataFrame({k: metrics(vals[k]) for k in vals}).T
for name, led in ledgers.items():
    if len(led) == 0:
        perf.loc[name, ["premium_paid_pa", "recovered_pa",
"net_hedge_cost_pa", "share_time_hedged"]] = 0
        continue
    led.to_csv(OUT / f"ledger_{SLUG[name]}.csv", index=False)
    pv = vals[name].reindex(led.date).values
    led["cash_pct"] = led.cash.values / pv * 100
    paid = -led.loc[led.cash < 0, "cash_pct"].sum()
    rec_ = led.loc[led.cash > 0, "cash_pct"].sum()
    perf.loc[name, "premium_paid_pa"] = paid / yrs
    perf.loc[name, "recovered_pa"] = rec_ / yrs
    perf.loc[name, "net_hedge_cost_pa"] = (paid - rec_) / yrs
    perf.loc[name, "share_time_hedged"] = (runs[name].notional_hedged >
0).mean() * 100
```

- The first line applies `metrics` to every column; `.T` transposes so strategies are rows.

- For each strategy with trades, the ledger is saved, then every cash flow is divided by the portfolio value on that date (`reindex(led.date)` looks the values up) to express it as a % of the portfolio.
- `paid` sums the negative cash flows (premium spent) and flips the sign; `rec_` sums the positive ones. Both are divided by the number of years for per-year figures.
- `share_time_hedged` is the share of days with any long put open.

Cost and payoff by regime

```
for name in list(STRATS)[1:]:
    led = ledgers[name].copy()
    pv = vals[name].reindex(led.date).values
    led["cash_pct"] = led.cash.values / pv * 100
    for basis in ("open_regime", "regime"):
        g = led.groupby(basis)
        for reg in ["Low", "Normal", "High"]:
            if reg not in g.groups:
                regime_rows.append(dict(strategy=name, basis=basis,
regime=reg, paid=0.0, received=0.0, net=0.0, n_trades=0))
                continue
            x = g.get_group(reg)
            paid = -x.loc[x.cash < 0, "cash_pct"].sum(); recv = x.loc[x.cash
> 0, "cash_pct"].sum()
            regime_rows.append(dict(strategy=name, basis=basis, regime=reg,
paid=paid, received=recv, net=recv - paid,
                                    n_trades=int((x.action ==
"open").sum())))
```

- `list(STRATS)[1:]` skips Unhedged.
- Two ways of grouping the ledger. `open_regime` gives each cash flow to the regime the option was **bought** in, so an option's whole life lands in one bucket. `regime` gives each cash flow to the regime on the day it **happened**.
- If a strategy never traded in a regime (the timed ones never buy in Normal or High), a row of zeros is written so the table stays complete. Otherwise, premium paid, money received and the net are summed, and `n_trades` counts the "open" rows.
- The totals are cumulative over the 6.28 years. The report and chart divide by years to get per-year figures.

```
dr = vals.pct_change()
bt["regime_prev"] = bt.regime.shift()
excess = dr.sub(dr["Unhedged"], axis=0).join(bt.regime_prev)
regime_excess = (excess.groupby("regime_prev", observed=True).sum() *
100).drop(columns="Unhedged")
days_in = excess.groupby("regime_prev", observed=True).size()
regime_excess_pa = regime_excess.div(days_in, axis=0) * TRADING_DAYS
```

The day-by-day view.

- `dr` holds every strategy's daily return. `dr.sub(dr["Unhedged"], axis=0)` subtracts the unhedged return from every column, row by row: the hedge's contribution each day.
- `regime_prev` is yesterday's regime, the one known when the day starts, so the grouping uses no future information.
- Summing by regime gives the cumulative contribution in each regime; dividing by the number of days in that regime and multiplying by 252 annualises it. This is where "−2.1% a year on low-VIX days, +2.0% on normal days, +6.3% on high-VIX days" comes from.

Stress episodes

```
ep = pd.DataFrame({k: (vals.loc[b] / vals.loc[a] - 1) * 100 for k, (a, b) in
                   ((k, (vals.index[vals.index.get_indexer([pd.Timestamp(a)],
method="bfill")[0]],
                         vals.index[vals.index.get_indexer([pd.Timestamp(b)],
method="ffill")[0]])) for k, (a, b) in episodes.items())}).T
```

The densest line in the script. Read it from the inside out.

- `episodes` maps each name to a (start, end) pair of dates.
- `get_indexer([date], method="bfill")` finds the first trading day on or after the start date; `method="ffill"` finds the last trading day on or before the end date. This snaps calendar dates to real trading days.
- The inner generator produces (name, (start_day, end_day)) pairs. The outer dictionary comprehension computes each strategy's return from start to end: `vals.loc[b] / vals.loc[a] - 1` divides two rows, giving one return per strategy.
- `.T` makes episodes the rows.

Trade summary of the proposal

```
L = ledgers[PROPOSED]
tr = L.groupby("trade_id").agg(open_date=("date", "first"), open_vix=("vix",
"first"), cash=("cash", "sum"),
                               exit=("action", lambda a:
",".join(sorted(set(a) - {"open"}))))
```

Groups the ledger by option: the date and VIX when bought, the net cash over its life, and how it ended (the set of its actions minus "open", joined into text such as "monetise_1,monetise_trail"). From this come the 22 trades, 5 winners, 3 monetised, and the list of monetisation sales.

Charts

- **Figure 6** plots every strategy's value (top) and drawdown (bottom), with the proposal and unhedged drawn thicker (`lw=2.0`).
- **Figure 7** is a grouped bar chart of net option P&L per year by the regime at purchase. With six strategies per group, `xx + (i − 2.5) × w` offsets bar i so the six bars sit side by side, centred on each regime's tick.

9. Robustness grids, the heatmap and saving results (lines 654 to 725)

Every robustness test is the same pattern: loop over values of one or two settings, call `run()` with them, compute `metrics()` , collect a row.

```
grid = []
for low in [12, 13, 14, 15, 16, 1e9]:
    for s1 in [20, 22, 25, 30]:
        o, _ = run("timed_hybrid", low=low, spike1=s1)
        m = metrics(o.value)
        grid.append(dict(buy_below=str(int(low)) if low < 100 else "always",
spike1=s1, CAGR=m["CAGR"], maxDD=m["maxDD"],
                         sharpe=m["sharpe"], calmar=m["calmar"]))
```

- 6 buy thresholds × 4 first-sale triggers = 24 backtests of the proposal. `1e9` means "always buy" and is labelled that way.
- `o, _ = run(…)` keeps the daily values and throws away the ledger (`_`).
- Saved as `sensitivity_thresholds.csv` and drawn as the heatmap in Figure 8.

```
for sk in [0.0, 0.6, 1.0]:
    for hs in [0.0, 0.5, 1.0]:
        for strat in ["always_put", "timed_hybrid"]:
            o, _ = run(strat, skew=sk, half_spread=hs)
```

Pricing stress test: 3 skew levels × 3 spreads × 2 strategies = 18 runs. Comparing the always-on put with the proposal under the same assumptions shows whether the timing advantage survives costlier options. It does: the proposal returned 12.9% to 15.1%, the always-on put 6.0% to 11.1%.

```
for ten in [1, 2, 3, 6]:
    o, _ = run("timed_hybrid", tenor=ten)
for mny in [0.90, 0.95, 1.00]:
    o, _ = run("timed_hybrid", moneyness=mny)
```

Tenor (1, 2, 3, 6 months) and strike (10% OTM, 5% OTM, ATM), one at a time. These rows are appended to the same table, `sensitivity_pricing.csv`.

```
RULES = {"Hold to expiry": "timed_hold", "Sell all at VIX 22": "timed_all",
"Ladder 22 / 30": "timed_ladder",
         "Trailing (fade 20%)": "timed_trail", "Hybrid (proposed)":
"timed_hybrid"}
for ten, tr, st, lab in [(2, 1, False, "2M rolled"), (3, 1, False, "3M
rolled"), (3, 1 / 3, True, "3M staggered thirds"),
                         (6, 1 / 6, True, "6M staggered sixths")]:
    for rn, rs in RULES.items():
        o, _ = run(rs, tenor=ten, tranche=tr, stagger=st)
        m, mp = metrics(o.value), metrics(o.value.loc["2020-07-01":])
        mat.append(dict(structure=lab, rule=rn, CAGR=m["CAGR"],
maxDD=m["maxDD"], sharpe=m["sharpe"],
                        covid=(o.value.asof(pd.Timestamp("2020-03-23")) /
o.value.asof(pd.Timestamp("2020-01-17")) - 1) * 100,
                        CAGR_ex_covid=mp["CAGR"],
maxDD_ex_covid=mp["maxDD"]))
```

The most revealing test: 4 structures × 5 monetisation rules = 20 runs.

- Each structure is a tuple (tenor, tranche size, stagger flag, label). "3M staggered thirds" buys a 3-month put on a third of the portfolio each eligible month and keeps the older ones, so up to three overlap.
- Each run is measured twice: over the full period (`m`) and from 1 July 2020 (`mp`), which drops the COVID crash.
- `covid` is the return from 17 January to 23 March 2020, read with `.asof()`.

- This table is where you can see that the full-period ranking depends on COVID, while the post-COVID CAGRs of all 20 runs sit between 13.8% and 15.3% a year.

```
post = vals.loc["2020-07-01":]
RESULTS["ex_covid_performance"] = {k: {kk: round(float(vv), 3) for kk, vv in
metrics(post[k]).items()} for k in post}
```

The main seven strategies re-measured from July 2020, a nested dictionary comprehension: for each strategy `k`, a dictionary of its rounded metrics. This gives the "CAGR from July 2020" column (15.4% unhedged against 14.9% for the proposal).

Figure 8, the heatmap.

```
h = grid.pivot(index="buy_below", columns="spike1", values=col)
h = h.reindex(sorted(h.index, key=lambda x: 1e9 if x == "always" else
float(x)))
good = h.values
norm_ = (good - good.min()) / (np.ptp(good) + 1e-9)
a.imshow(norm_, cmap="Blues", aspect="auto", vmin=-0.15, vmax=1.15)
```

- `pivot` reshapes the long grid into a matrix: buy thresholds down the side, triggers across the top.
- The thresholds are text ("12" … "always"), so they are sorted with a key that puts "always" last.
- `norm_` rescales the values to 0–1 (`np.ptp` is max minus min; `1e-9` avoids dividing by zero). Higher CAGR and shallower drawdown (less negative) both give larger values, so darker always means better. An earlier version coloured drawdowns the wrong way round; `good = h.values` is the fix.
- `imshow` draws the colour grid. `vmin` / `vmax` slightly beyond 0–1 keep the extremes from being pure white or black. The loop after it writes each number in its cell, white on dark cells and black on light ones.

Saving everything.

```
with open(OUT / "results.json", "w") as f:
    json.dump(RESULTS, f, indent=1, default=str)
print("done")
```

`json.dump` writes the `RESULTS` dictionary to a file. `default=str` converts anything JSON cannot store directly (like dates) to text. Every number in the report and the deck can be traced to a key in this file.